



QSOE

Quick and Secure Operating Environment

User Guide

Installing, booting, and using QSOE

Document: QSOE-USR

Revision: Rev A — Draft

Applies to: QSOE 0.1

Yuri Zaporozhets

June 2026

© 2026 Yuri Zaporozhets.

Licensed under CC BY-ND 4.0 — verbatim redistribution with attribution; no derivatives.

<https://creativecommons.org/licenses/by-nd/4.0/>

Contents

1	Introduction	1
1.1	What QSOE is, briefly	1
1.2	The two variants	1
1.3	What you need	1
1.4	A note on version 0.1	2
1.5	How to read this guide	2
2	Obtaining and Running QSOE	3
2.1	What to download	3
2.2	Prerequisites for QEMU	3
2.3	Running the image	4
2.4	Why two QEMU machines	4
2.5	Tuning the launch	5
2.6	On real hardware	5
3	Boot and Login	6
3.1	From power-on to a prompt	6
3.2	The boot menu	6
3.3	Init: from boot image to disk	6
3.4	The login banner and prompt	7
3.5	The supplied accounts	7
3.6	After login	8
4	The Shell (qsh)	9
4.1	Interactive and login shells	9
4.2	The prompt	9
4.3	Editing the command line	9
4.4	Command history	10
4.5	Built-in commands	10
4.6	Running programs and jobs	10
4.7	Scripting	11
4.8	Customizing your shell: <code>.profile</code>	11
5	Commands and Utilities	12
5.1	Files and directories	12
5.2	Processes and the system	12
5.3	Clock and timing	13
5.4	Diagnostics	13
5.5	Powering off and rebooting	13
5.6	The test suite	14

6	The Filesystem and System Views	15
6.1	One namespace, many servers	15
6.2	/usr: the on-disk filesystem	15
6.3	Read-only today, writable tomorrow	16
6.4	/dev: devices	16
6.5	/sys: system facts	16
6.6	/proc: running processes	17
7	Files and Editing (Forthcoming)	18
7.1	QSOE Commander	18
7.2	The built-in editor	18
7.3	Why this matters for QSOE	18
8	Troubleshooting and Shutdown	20
8.1	When boot does not reach login: the rescue shell	20
8.2	Reading the system log	20
8.3	Common early problems	21
8.4	Shutting down	21
8.5	Where to go next	21

Chapter 1

Introduction

This guide is for someone sitting in front of QSOE: it explains how to get the system, boot it, log in, and use the shell, the commands, and the filesystem. It does not assume you have read anything else about QSOE first.

1.1 What QSOE is, briefly

QSOE — the Quick and Secure Operating Environment — is a small, QNX-style operating environment for 64-bit RISC-V. Like QNX, it keeps the kernel tiny and runs almost everything else — drivers, the filesystem, the shell — as ordinary userspace programs that talk to one another by passing messages. For a user that structure mostly stays out of sight; where it shows through, this guide points it out.

What makes QSOE unusual is that it is really *two* systems that share one userspace. The same shell, the same commands, and the same libraries run, unchanged, on either of two interchangeable microkernels. You can boot whichever you like; once you are at the shell prompt they look and behave the same. The *why* of that design is the subject of a separate manual, the *Design & Architecture* guide (QSOE-DSN); here we care only about how it affects what you do.

1.2 The two variants

When you boot QSOE you choose one of two variants:

- **QSOE/N** — QSOE on **Skimmer**, the project’s own from-scratch microkernel.
- **QSOE/L** — QSOE on **seL4**, the formally verified capability microkernel.

The published disk image carries both, and the boot menu lets you pick one at power-on (Chapter 2). The difference is underneath the system, not in front of it: the same login, the same **qsh** shell, and the same commands are there either way. Throughout this guide, anything that genuinely differs between the two is called out explicitly; everything else applies to both.

1.3 What you need

To run QSOE under emulation you need a RISC-V QEMU and the UEFI firmware for it; the exact packages and the launch command are in Chapter 2. To run it on real hardware you need a supported board — today the SiFive HiFive Unmatched — covered briefly in the same chapter. Either way, the system you boot is the same self-booting image.

1.4 A note on version 0.1

This guide describes QSOE 0.1, the first public release. QSOE is young, and several pieces are deliberately on the near horizon rather than in your hands today — a writable in-memory filesystem, a two-panel file manager with a built-in editor, multi-console login. Where this guide describes something that is planned rather than present, it says so plainly and marks it:

Planned content. A feature flagged like this is on the roadmap, not in 0.1.

Everything not so marked is in the 0.1 system and works as described. This keeps the guide honest about where the system is while still showing where it is going.

1.5 How to read this guide

The chapters run roughly in the order you will meet the system:

- Chapter 2 — obtaining QSOE and running it under QEMU (and a word on real hardware).
- Chapter 3 — what happens from power-on to the login prompt, and logging in.
- Chapter 4 — the `qsh` shell.
- Chapter 5 — the commands and utilities that ship with the system.
- Chapter 6 — the filesystem and the `/sys` and `/proc` views.
- Chapter 7 — the forthcoming file manager and editor.
- Chapter 8 — the rescue shell, the system log, and shutting down.

Chapter 2

Obtaining and Running QSOE

The quickest way to meet QSOE is to boot the published disk image under QEMU. The image is *self-booting*: it carries its own bootloader and both kernels, so you do not pass a kernel on the command line — you boot the disk exactly as a machine would, and a menu lets you pick the variant. This chapter covers what to download, how to run it under emulation, and — briefly — how the same image boots on real hardware.

2.1 What to download

The download area carries two kinds of artifact:

- **The disk image** — a single self-booting image containing the mr-bml bootloader, both kernels (QSOE/N and QSOE/L), and the shared userspace on a filesystem. A second, small companion image (`virtio.img`) accompanies it for QSOE/L; the reason is explained in §2.4.
- **The manuals** — this guide and its companions, as PDFs.

Alongside the image is the launch script `run-nvme.sh`, which starts QEMU with the right options for either variant. You can of course write your own QEMU command line, but the script encodes a number of small, non-obvious details (§2.4) that are easy to get wrong, so it is the recommended way to boot the image.

2.2 Prerequisites for QEMU

To boot the image under emulation you need two things on your host:

- **A RISC-V system emulator** — `qemu-system-riscv64`.
- **UEFI firmware for RISC-V** — the edk2 build shipped as the `qemu-efi-riscv64` package, which provides the firmware files `RISCV_VIRT_CODE.fd` and `RISCV_VIRT_VARS.fd`. The firmware is what runs the bootloader off the disk, just as it would on a physical board.

On a Debian-based host both come from the distribution’s own packages. If your firmware lives somewhere unusual, you can point the launcher at it with the `EDK2_CODE` and `EDK2_VARS` environment variables (§2.5).

Mind the QEMU version. QSOE/N requires **QEMU 11.0.1 or newer**. Its NVMe disk is driven by message-signalled interrupts on QEMU’s AIA interrupt controller, and versions up to and including 11.0.0 carry a bug that keeps such an interrupt from ever

reaching the kernel — the NVMe device initializes and then hangs. The fix landed in the 11.0.1 release. QSOE/L, which uses the older PLIC interrupt controller, is not affected; but since the common case is to try QSOE/N first, treat 11.0.1 as the floor. If your distribution's QEMU is older, build a current one and point the launcher at it with `QEMU=/path/to/qemu-system-riscv64`.

2.3 Running the image

With the image and `run-nvme.sh` in the same directory, booting is a single command. Run it with no argument for an interactive menu:

```
$ ./run-nvme.sh
Boot the QSOE NVMe image under QEMU:
  1. Run QEMU suitable for QSOE/N (Skimmer, NVMe root)
  2. Run QEMU suitable for QSOE/L (seL4, virtio root)
-->
```

or name the variant directly to skip the menu:

```
$ ./run-nvme.sh n      # boot QSOE/N (Skimmer)
$ ./run-nvme.sh l      # boot QSOE/L (seL4)
```

QEMU then runs the firmware, the firmware runs the mr-bml bootloader off the disk, and mr-bml shows the QSOE boot menu over the serial console. From there the boot proceeds as Chapter 3 describes. The console is the terminal you launched from — mr-bml, the kernel, and the shell all speak over it — so you interact with the system right there; no graphical window is opened by default. If you would rather also have edk2's graphical console, set `GRAPHIC=1` in the environment.

A faster path for source builds. If you built QSOE from source rather than downloading it, `make nvme` produces the same image and the in-tree launcher boots it identically. But there is also a quicker way: each kernel's tree has its own `emu.sh` (`nq/emu.sh`, `lq/emu.sh`) that boots the kernel *directly* — handing QEMU the kernel with `-kernel` and skipping the firmware, the bootloader, and the disk image entirely. That removes more than ten seconds of boot per launch, which adds up during development:

```
$ ./nq/emu.sh          # boot QSOE/N directly (no firmware, no mr-bml)
$ ./lq/emu.sh          # boot QSOE/L directly
```

These per-kernel launchers apply the same QEMU-version check as above. The full-image path (`run-nvme.sh`) is what reproduces a real boot, firmware and bootloader included; the direct `emu.sh` path is the fast inner loop while developing. Both reach the same shell.

2.4 Why two QEMU machines

You may notice that the launcher uses a slightly different QEMU machine for each variant. This is a limitation of QEMU's RISC-V emulation, not of QSOE, and the script handles it for you — but it is worth understanding why the two look different from the host side:

- **QSOE/N** roots its main filesystem on an emulated NVMe disk. NVMe delivers interrupts by PCIe message signalling (MSI-X), which on QEMU's `virt` board needs the modern AIA interrupt controller. So QSOE/N runs on a `virt` machine with AIA enabled.
- **QSOE/L** runs on stock seL4, which does not (yet) drive AIA, so it uses the classic PLIC `virt` machine. Without NVMe there, QSOE/L takes its main filesystem from a

small *virtio* disk instead — this is the companion `virtio.img`. The NVMe image is still attached, because it is what carries the bootloader and the kernels.

In both cases the firmware is told to forward the machine’s real device tree to the system (the launcher’s `acpi=off`); QSOE’s kernels discover their hardware from that device tree, and need it rather than the firmware’s ACPI tables. None of this changes what you see once you are at the shell — it is purely about how the emulated machine is wired up underneath.

2.5 Tuning the launch

The launcher reads a few environment variables so you can adjust the run without editing it. The most useful:

Variable	Effect
NVME_IMG, VIRTIO_IMG	paths to the disk images
EDK2_CODE, EDK2_VARS	paths to the UEFI firmware files
N_CPUS, N_MEM	cores and memory for QSOE/N
L_CPUS, L_MEM	cores and memory for QSOE/L
GRAPHIC	1 also opens a graphical window
QEMU	a different QEMU binary

Table 2.1: The main launch overrides.

For example, to boot QSOE/N with more cores and point at a downloaded image in the current directory:

```
$ NVME_IMG=./nvme.img N_CPUS=4 ./run-nvme.sh n
```

2.6 On real hardware

The same disk image boots on a physical board. QSOE’s bootloader, `mr-bml`, is a UEFI application: written to the image’s EFI System Partition, it is what the board’s firmware loads, and it presents the same menu and boots the same kernels as under QEMU. The supported board today is the **SiFive HiFive Unmatched** (FU740), where QSOE/N roots on a real NVMe SSD exactly as the emulated QSOE/N does.

Writing the image to a disk and booting a board is beyond a first run under QEMU, and the mechanics differ by board; for 0.1, treat real-hardware boot as “the same image, written to a real disk.” Everything in the rest of this guide — login, the shell, the commands — is identical whether you reached the prompt under QEMU or on the Unmatched.

Chapter 3

Boot and Login

This chapter walks through what happens between power-on and a usable shell: the boot menu, the start-up messages that scroll past, the login banner and prompt, and the accounts you can log in as. None of it is something you have to drive — the system gets itself to a login prompt on its own — but knowing what each stage is makes the console far less mysterious the first time, and tells you where to look if a boot does not reach the prompt (Chapter 8).

3.1 From power-on to a prompt

Reaching the shell is a short relay, each stage handing off to the next:

1. **The bootloader (mr-bml)** shows a menu and lets you pick a variant (§3.2).
2. **The kernel** — Skimmer or seL4 — starts and brings up **taskman**, the central system server.
3. **Init** runs: a first stage from the boot image brings up the bare essentials and mounts the on-disk filesystem, then hands off to a second stage on disk (§3.3).
4. **Login** starts: a banner, a **login:** prompt, and a password (§3.4).

The whole sequence takes only seconds under QEMU. The console you launched from carries all of it — bootloader menu, kernel and init messages, and the login prompt all appear there.

3.2 The boot menu

The first thing you see is mr-bml, QSOE’s bootloader, presenting a short menu with one entry per variant — **QSOE/N** and **QSOE/L**. Choose one and it loads that variant’s kernel and userspace and starts it; with no choice it boots the default after a brief timeout. This is the same menu whether you booted under QEMU or from a real disk, because it is the same bootloader on the same image.

Picking a variant here is the one moment the two-kernel nature of QSOE is visible to you. After this point the systems converge: everything the rest of this chapter describes looks the same on both.

3.3 Init: from boot image to disk

Once the kernel and **taskman** are up, the first userspace program, **/sbin/init**, runs. On a first boot you will see it narrate its work, a line per step — this is normal, and worth recognizing:

```
[init] starting slogger...
[init] starting pci-server...
[init] syncing system clock from hardware...
[init] starting devc-ser8250...
[init] repointing /dev/console -> /dev/ser1...
[init] starting devb-nvme...
[init] mounting /dev/nvme0n1p8 at /usr...
```

Read top to bottom, those lines are the system assembling itself: the logging service, the PCI bus, the wall clock seeded from the hardware real-time clock, the interrupt-driven serial console (after which the console is handed over from the kernel’s basic debug output to the real driver), and finally the storage driver and the root filesystem mounted at `/usr`. The exact device names can vary, but only under QEMU: there QSOE/N mounts an NVMe partition while QSOE/L mounts a virtio disk (§2.4). On real hardware — the Unmatched — there is no such split: both variants root on the same NVMe SSD. Either way the shape is identical.

This first init comes from the boot image and does only the minimum. Having mounted `/usr`, it hands off to a second-stage init that lives *on disk*, `/usr/sbin/sysinit/level1.sh`, which is where system start-up continues and which can be edited without rebuilding the boot image. The second stage finishes by starting login on the console.

If the disk cannot be mounted. If there is no usable root filesystem — an empty or unformatted disk, say — init does not stall; it drops you to an interactive rescue shell straight from the boot image, so you can still look around. That path is covered in Chapter 8.

3.4 The login banner and prompt

System start-up ends by starting `getty` on the console, which prints a one-line banner and then prompts you to log in. The banner identifies the running system and is assembled live from the `/sys` model (§6.5), not baked in — so it always reflects the variant and build you actually booted:

```
QSOE/N v0.17 (2026-06-28) on riscv-virtio,qemu

login:
```

The `osname` (QSOE/N on Skimmer, QSOE/L on seL4), the version and build date, and the board name all come from `/sys`; the same facts are what a `uname`-style tool reads (§6.5). The version in the banner is the running kernel’s own component version (here `v0.17`), which advances independently of — and so will not match — the umbrella release number this guide is written against (QSOE 0.1). QSOE numbers each component (the two kernels, the userspace, the C library) separately, and an umbrella release such as 0.1 simply bundles a tested set of those component versions; the banner reports the component, the download reports the bundle. Type a user name at `login:` and press Enter, and `login` asks for a **Password:** (the password is not echoed as you type). Authenticate successfully and you are dropped into your shell; get it wrong and you get `Login incorrect` and another try, after a short pause.

3.5 The supplied accounts

QSOE 0.1 ships with two accounts, defined in the system’s `passwd` and `shadow` files (Table 3.1). They are conventional Unix accounts: a superuser and an unprivileged user.

These are the default passwords of the published 0.1 image; on a system you intend to keep, change them. Both accounts use `qsh` as their login shell. `root`’s home is the filesystem root; `user` has an ordinary home under `/home`. For everyday use, log in as `user`; switch to `root` only

Login	Password	uid	Home	Role
root	QSOE	0	/	superuser
user	skimmer	1000	/home/user	ordinary user

Table 3.1: The accounts shipped in QSOE 0.1, with their default passwords.

when you need the superuser’s unrestricted access. The privilege rule QSOE enforces is the conventional one — an unprivileged process cannot promote itself to `root` — and is described in the *Design & Architecture* guide.

3.6 After login

A successful login does three things before handing you the shell: it switches to your account’s user and group identity, moves to your home directory, and starts your login shell. Because it is a *login* shell, `qsh` sources your `~/.profile` on the way in, which is where per-user settings live. The stock `user` profile sets a file-creation mask and prints a short greeting:

```
Welcome to QSOE, user.
[/home/user]$
```

From that prompt the system is yours to use. The prompt itself — the current directory in brackets, then `$` for an ordinary user or `#` for `root` — and the shell behind it are the subject of Chapter 4.

One console, for now. QSOE 0.1 runs login on the single system console. When you log out, start-up re-prompts so the console stays usable. **Planned content.** Multiple virtual consoles, and a `getty` that owns its own line (driven by an `inittab`-style configuration), are planned; 0.1 is deliberately the single-console case.

Chapter 4

The Shell (qsh)

The shell is the program you spend the most time with, and on QSOE that shell is **qsh**. It is a Korn-style shell — derived from the MirBSD Korn shell (mksh) — so if you know **ksh** or **bash**, you already know **qsh**: the same command syntax, the same variables and quoting, the same control structures and functions. This chapter covers using it interactively — the prompt, line editing, history, the built-in commands, and job control — and then its use for scripting.

qsh is also `/bin/sh`: it is the interpreter for the system’s own start-up scripts (Chapter 3), so the shell you type at and the shell that boots the system are one and the same.

4.1 Interactive and login shells

When you log in, **login** starts **qsh** as a *login* shell, which means it first sources your `~/.profile` (§4.8). After that it is an ordinary *interactive* shell: it prints a prompt, reads a command line, runs it, and repeats. You can also start further interactive shells by simply running **qsh**, and run a script non-interactively by giving it a file (§4.7).

4.2 The prompt

The default prompt shows your current directory in brackets, followed by a `$` for an ordinary user or a `#` when you are **root**:

```
[/home/user]$ cd /sys  
[/sys]$
```

The prompt is just the value of the `PS1` variable, in the familiar backslash-escape style: `\w` expands to the working directory and `\$` to the `#/$` sign. Set `PS1` in your `.profile` to taste.

4.3 Editing the command line

qsh has a compact built-in line editor — always on, with no mode to select. The key bindings are the usual control-key set (Table 4.1); if you have used **emacs**-style editing in another shell, these will feel immediate.

The editor runs its own echo and cursor handling directly, because the QSOE serial console delivers raw bytes with no kernel line discipline — so what you see is the shell’s own doing, not the terminal’s.

Planned content. TAB currently inserts a literal tab; filename, command, and variable completion on TAB is planned.

Keys	Action
<code>^A</code> / <code>^E</code>	move to start / end of line
<code>^B</code> / <code>^F</code>	move back / forward one character
<code>^H</code> , Backspace, DEL	delete the character before the cursor
<code>^W</code>	delete the word before the cursor
<code>^U</code> / <code>^K</code>	kill to start of line / to end of line
<code>^P</code> / <code>^N</code> , <code>↑</code> / <code>↓</code>	previous / next history line
<code>^R</code>	incremental reverse history search
<code>^L</code>	redraw the line
<code>^D</code> on an empty line	end of input (logout)

Table 4.1: The `qsh` line-editor key bindings.

4.4 Command history

Every command you enter is remembered for the session. Recall earlier commands with `^P/^N` or the arrow keys, search back through them with `^R`, and list or re-run them with the `fc` built-in (`fc -l` lists recent history; `fc -e` - re-runs the last command).

History is kept in memory for the running session. Carrying history across logins uses the `HISTFILE` variable, as in any Korn shell — but in 0.1 the root filesystem is read-only (§6.1), so there is nowhere to write it yet.

Planned content. Persistent, cross-session history arrives with the writable filesystem (§6.3).

4.5 Built-in commands

Some commands are built into the shell itself rather than being separate programs — because they must change the shell’s own state (like `cd`), or because they are part of the language (like `test`). Table 4.2 lists what `qsh` provides; their behavior is the standard Korn-shell one.

Group	Built-ins
Navigation	<code>cd</code> (<code>chdir</code>), <code>pwd</code> , <code>realpath</code>
Variables	<code>typeset</code> , <code>export</code> , <code>readonly</code> , <code>unset</code> , <code>set</code> , <code>shift</code>
Aliases	<code>alias</code> , <code>unalias</code>
I/O	<code>print</code> , <code>echo</code> , <code>read</code>
Control flow	<code>.</code> (<code>source</code>), <code>eval</code> , <code>exec</code> , <code>exit</code> , <code>return</code> , <code>break</code> , <code>continue</code> , <code>trap</code>
Conditionals	<code>test</code> (<code>[]</code>), <code>true</code> (<code>:</code>), <code>false</code>
Arithmetic	<code>let</code>
Arguments	<code>getopts</code>
Jobs	<code>jobs</code> , <code>wait</code> , <code>kill</code> (§4.6)
Lookup/limits	<code>whence</code> , <code>command</code> , <code>builtin</code> , <code>fc</code> , <code>umask</code> , <code>ulimit</code> , <code>times</code> , <code>id</code>

Table 4.2: `qsh` built-in commands.

4.6 Running programs and jobs

To run a program, type its name; the shell looks it up along `PATH` (set for you to the system directories) and runs it. Everything else — the commands and utilities in Chapter 5 — is reached this way.

You can run a command in the background by ending it with `&`, list background jobs with `jobs`, wait for them with `wait`, and signal them with `kill`:

```
[/home/user]$ suite &           # start a long task in the background
[/home/user]$ jobs              # see what is running
[/home/user]$ kill %1          # signal job number 1
```

Planned content. Suspending and resuming foreground jobs (`^Z`, `fg`, `bg`) is not part of 0.1; background (`&`), `jobs`, `wait`, and `kill` are.

4.7 Scripting

Because `qsh` is a full Korn-style shell, scripting is exactly what you would expect: variables and parameter expansion, `if/while/for/case`, functions, command substitution, pipelines, and the `test` built-in for conditionals. A script is an ordinary text file with a `#!/bin/sh` line; make it runnable and run it by name.

You do not have to take this on faith: the system's own start-up is written in `qsh`. The boot scripts of Chapter 3 — `/sbin/init` and the on-disk `/usr/sbin/sysinit/level1.sh` — are real, readable `qsh` scripts that bring the whole system up, and they are a good worked example of the dialect in practice.

Note on what is absent. QSOE has no `fork()`: every external command and every pipeline stage is started with `posix_spawn()` under the hood. This is invisible at the script level — pipelines, subshells, and background jobs all work as written — but it is why a few exotic `fork`-specific tricks from other shells do not apply. The reasoning is in the *Design & Architecture* guide.

4.8 Customizing your shell: `.profile`

Your login shell sources `~/.profile` at start, so that is where to put per-user settings: environment variables, aliases, and a custom prompt. The `user` account ships with a deliberately minimal one — it sets a file-creation mask and prints a greeting:

```
umask 022
echo
echo "Welcome to QSOE, user."
echo
```

Add your own exports and aliases here. Note that in 0.1 the home directory is on the read-only root filesystem (§6.1), so edits to `.profile` are for when the writable filesystem lands (§6.3); until then it is best read as the example it is.

Chapter 5

Commands and Utilities

Beyond the shell's own built-ins (§4.5), QSOE 0.1 ships a small set of stand-alone commands: file tools, process and system inspection, clock and timing, diagnostics, and power control. They are deliberately few — 0.1 is a foundation release — but they cover looking around a running system and managing it. This chapter is a short reference, grouped by what you would reach for them to do.

Every command supports a terse usage reminder, and each does one thing; the shell finds them along your `PATH`, so you invoke them by bare name.

5.1 Files and directories

ls — **list directory contents**. The familiar listing tool. `ls -l` gives the long form (type, permissions, size, name) and `ls -a` includes dotfiles:

```
[/home/user]$ ls -la /  
[/home/user]$ ls /sys
```

cat — **concatenate files to standard output**. Standard POSIX `cat`: with no arguments (or a single `-`) it reads standard input, and it is binary-safe. It is the quickest way to read a short file — including the text files under `/sys` and `/proc` (Chapter 6):

```
[/home/user]$ cat /sys/version  
[/home/user]$ cat /proc/1/info
```

5.2 Processes and the system

ps — **process, thread, and interrupt status**. Reports what is running. By default `ps` prints one terse line per process (pid, state, time, command), in the spirit of the BSD `ps`; `ps -H` expands each process to show its threads, with single-letter thread states. Its data comes from `/proc` and the kernel's thread table, so it sees not just processes but the threads, timers, and interrupt handlers behind them.

sysinfo — **the whole machine at a glance**. A one-shot report that composes, into a single printout, what other systems scatter across several tools: the board and command line, the RAM range and memory map, the CPUs with a sampled per-hart load, the interrupt controller, and the PCI geometry. Run it with no arguments when you want the lay of the land.

syscmd — system-management multi-tool. A single binary that wraps the small calls a script (or you) might make to manage the system, one per *verb*. The user-facing verbs are **reopen** (point `/dev/console` at another path) and the power verbs covered in §5.5. Collecting these as verbs of one tool is deliberate — new system actions become new **syscmd** verbs rather than a scatter of one-off binaries.

5.3 Clock and timing

rtc — the battery-backed hardware clock. With no argument, **rtc** shows both the hardware clock and the system clock. **rtc hw** sets the *system* clock from the hardware clock (this is what `init` runs early in boot, so every later timestamp is true Unix time), and **rtc date** sets the *hardware* clock from the system clock. Setting a clock is restricted to **root**.

time — how long a command takes. There are, in 0.1, two different **times**, and it is worth knowing which one you get:

- **The shell reserved word.** Writing **time** at the start of a command invokes **qsh**'s built-in timing keyword (as in any Korn shell), *not* a program — the shell recognizes **time** before it ever looks along **PATH**. It prints real, user, and system times in the traditional shell format. But QSOE does not account per-process CPU time, so the user and system figures are always zero; only the real (wall-clock) figure is meaningful.
- **The `/usr/bin/time` program.** The stand-alone utility reports just the elapsed real time — no empty user/system columns — and passes the command's exit status straight through, so it stays transparent in a pipeline. Because the reserved word shadows it, you reach the program by its full path:

```
[/home/user]$ time ls /usr          # the shell keyword (zeroed user/sys)
[/home/user]$ /usr/bin/time ls /usr # the program (real time only)
```

Planned content. The two will be unified in 0.2 — either by dropping the shell's built-in **time** or by folding the program's behavior into it — so that a bare **time** does the useful, unsurprising thing.

5.4 Diagnostics

sloginfo — read the system log. The system logging service (**slogger**, started first in boot) collects log events from the kernel and from servers; **sloginfo** drains and prints them in human-readable form, one event per line with a timestamp, a severity, the reporting subsystem, and the message. It is the first place to look when something has not started as expected (Chapter 8).

lspci — list PCI devices. Lists the devices on the PCI bus, one line each. **lspci -v** adds the subsystem, interrupt, and sized BARs; **lspci -vv** adds a raw configuration-space dump. The command **pci** is a synonym (a link to the same tool).

5.5 Powering off and rebooting

shutdown — power the machine off. Brings the system down and powers it off. **reboot** instead cold-restarts the machine. Both are restricted to **root**, and both are thin links to

`syscmd`'s power verbs (§5.2); `shutdown` is also what the rescue shell suggests when there is nothing else to do (Chapter 8).

```
[/]# shutdown
```

5.6 The test suite

One more program is worth knowing, though it is not an everyday tool: `suite`, in `/usr/bin`, is the shared regression suite that exercises the system's IPC, synchronization, and library surface. The same suite runs on both QSOE/N and QSOE/L, and it is what keeps the “one userspace, two kernels” promise honest: a feature is not done until it passes on both. Running it is a quick way to confirm a system is healthy end to end.

Where commands live. The everyday commands above are in `/bin`, available from the moment the system boots — even in the rescue shell, before any disk is mounted. A few programs (`time`, `sysinfo`, `suite`) live in `/usr/bin` on the mounted filesystem, so they are present once `/usr` is up (§6.1). Your `PATH` spans both, so in normal use the distinction is invisible.

Chapter 6

The Filesystem and System Views

QSOE presents a single directory tree, the way any Unix does — but underneath, that one tree is assembled from several independent servers. A real on-disk filesystem supplies `/usr`; device drivers supply `/dev`; the system server itself supplies the `/sys` and `/proc` views. You navigate all of it with the same `cd`, `ls`, and `cat`; this chapter explains what each part is, so the layout makes sense and you know what you can write to (in 0.1, and later).

6.1 One namespace, many servers

There is one pathname space, shared by every process. When you open a path, the system server (`taskman`) resolves it: each server has registered the path prefix it is responsible for, and the open is routed to whichever server owns that prefix. Opening `/dev/ser1` reaches the serial driver; opening a file under `/usr` reaches the filesystem server; reading `/sys/version` reaches `taskman` itself. This is invisible in use — a path is a path — but it is why the tree can span a disk, several drivers, and the kernel's own bookkeeping without you having to think about where a name lives.

The top of the tree in 0.1 looks like this:

Path	Provided by	What it is
<code>/bin</code> , <code>/sbin</code>	boot image	core commands and servers
<code>/usr</code>	disk filesystem	the on-disk system (§6.2)
<code>/dev</code>	device drivers	devices (§6.4)
<code>/sys</code>	<code>taskman</code>	system facts (§6.5)
<code>/proc</code>	<code>taskman</code>	running processes (§6.6)
<code>/etc</code>	symlink	→ <code>/usr/conf</code>
<code>/home</code>	symlink	→ <code>/usr/home</code>

Table 6.1: The top-level namespace in QSOE 0.1.

The `/etc` and `/home` names are symlinks into `/usr`, so the familiar paths work (`/etc/passwd`, `~` under `/home`) while the actual files live on the one writable-in-future disk tree.

6.2 `/usr`: the on-disk filesystem

Everything that lives on disk is under `/usr`, served by `fs-qrv`, the filesystem server, from the main disk — an NVMe partition on QSOE/N (and on real hardware), a virtio disk on QSOE/L under QEMU (§2.4). Init mounts it early in boot (Chapter 3); from then on it holds:

- `/usr/bin`, `/usr/sbin` — programs that are not part of the minimal boot image: utilities such as `time` and `sysinfo`, the test suite, and the login programs `getty` and `login`.
- `/usr/conf` — system configuration, reached as `/etc`: the `passwd`, `shadow`, and `group` files.
- `/usr/home` — user home directories, reached as `/home`.
- `/usr/sbin/sysinit` — the on-disk, editable system start-up (`level11.sh`), which the boot-image `init` hands off to.

6.3 Read-only today, writable tomorrow

One thing to know up front about 0.1: **the filesystem is read-only**. The `fs-qrv` server mounts `/usr` for reading, and there is, in this release, no writable storage anywhere in the tree. You can read, run, and inspect everything; you cannot yet create or change a file. This is why several things elsewhere in this guide are deferred — saved shell history (§4.4), edits to your `.profile` (§4.8): there is simply nowhere to write them yet.

Planned content. A writable, in-memory filesystem (`ramfs`) is a required part of 0.2. It will provide ordinary read/write files *and* directories — notably, unlike QNX’s file-only `tmpfs`, it will let you create directories — giving the system scratch space for temporary files and a place that survives writes (until reboot). Whether the on-disk `fs-qrv` filesystem itself becomes writable by 0.2 is still open; the in-memory filesystem is the firm commitment.

Until then, treat 0.1 as a live, inspectable system you boot fresh each time rather than one you accumulate state on.

6.4 /dev: devices

Under `/dev` are the system’s devices, each served by its driver rather than by a filesystem. The ones you will meet:

- `/dev/console` — the system console; in normal operation it is bound to the interrupt-driven serial port (§3.3).
- `/dev/ser1` — the serial line itself, behind the console.
- `/dev/slog` — the system log, drained by `sloginfo` (§5.4).
- `/dev/null` — the usual bit-bucket.
- the disks — `/dev/nvme0n1p8` (QSOE/N) or `/dev/vb1k0` (QSOE/L), the partition `fs-qrv` mounts as `/usr`.

Because each device is a path served by a driver, the same `open`, `read`, and `write` you use on files work on devices — that is the point of representing them as paths.

6.5 /sys: system facts

`/sys` is a small, read-only view of what the system *is*, published by `taskman` (and `fed`, on QSOE/N, from a kernel-built information page). Each entry is a short text file you can simply `cat`:

These are the same facts the login banner is assembled from (§3.4) and that a `uname`-style query reads; `sysinfo` (§5.2) presents a fuller version of the same information. Reading the system’s identity this way — as ordinary files — rather than through a special system call is a deliberate part of QSOE’s design, described in the *Design & Architecture* guide.

Entry	Contents
<code>/sys/osname</code>	the variant name (QSOE/N or QSOE/L)
<code>/sys/version</code>	the running version and build date
<code>/sys/board</code>	the board / machine name
<code>/sys/cmdline</code>	the kernel command line

Table 6.2: Some `/sys` entries.

6.6 `/proc`: running processes

`/proc` is the live view of what the system *is doing*: one entry per running process, named by its process id. Each carries an `info` file with the process’s identity — pid, parent, state, and name — as plain text:

```
[/home/user]$ cat /proc/1/info
```

Process 1 is `taskman` itself, the root of everything. This is the same data `ps` (§5.2) reads and formats into columns; reading `/proc` directly is there for when you want the raw facts or want to read them from a script.

Chapter 7

Files and Editing (Forthcoming)

This chapter describes where QSOE’s everyday file handling is going. None of what follows is in 0.1 — it is presented here so the shape of the system to come is clear, and so this guide grows into it rather than gaining a whole new chapter later. Everything in this chapter is planned, not present.

7.1 QSOE Commander

The intended centerpiece of day-to-day file work is **QSOE Commander**, a two-panel file manager invoked as `qc`. If you have used Midnight Commander (`mc`) or its Norton Commander ancestor, the model is exactly that one: two side-by-side panels, each showing a directory, with one panel active at a time. You move with the arrow keys, switch panels with `Tab`, and drive the common operations — view, edit, copy, move, make a directory, delete — from the function keys along the bottom, acting between the two panels. It is a fast, keyboard-only way to get around a filesystem, and it is meant to be QSOE’s flagship interactive application: a single program that exercises the whole userspace — the filesystem, the terminal, processes, and the C library — end to end.

Planned content. QSOE Commander (`qc`) is planned for QSOE 0.4, the release built around it. It depends on the writable filesystem (§6.3) arriving first in 0.2 — copying, moving, creating directories, and saving edits all need somewhere to write.

7.2 The built-in editor

From within QSOE Commander, pressing `F4` on a file opens it in the built-in editor. The editor is modeled on `mcedit` — Midnight Commander’s own editor — in look and feel: a straightforward, full-screen text editor with the same kind of function-key interface as the file manager around it, rather than a modal editor in the `vi` tradition. It is meant for the everyday job of reading and changing a configuration file or a script without leaving the file manager you are already in.

Planned content. The `F4` editor ships as part of QSOE Commander. Like the file manager, saving requires the writable filesystem (§6.3); until that lands, editing has nowhere to store its result.

7.3 Why this matters for QSOE

A two-panel file manager with an integrated editor is a deceptively demanding program: it opens and reads directories, copies and moves files, spawns other programs, redraws a full-screen interface over the terminal, and edits text — all at once. Standing it up on QSOE is

therefore one of the best whole-system tests the project has, which is exactly why it anchors a release of its own. For you as a user, it is the point at which QSOE stops being only a shell-and-commands system and gains a comfortable, visual way to work with files.

Until then, the shell (Chapter 4) and the file commands (Chapter 5) are how you work with the filesystem — reading, listing, and running, on the read-only system of 0.1.

Chapter 8

Troubleshooting and Shutdown

Most of the time QSOE boots straight to a login prompt and there is nothing to troubleshoot. This last chapter is for the times it does not: how to read the system when a boot stops short, where the log is, the handful of problems you are most likely to hit early on, and — finally — how to bring the system down cleanly.

8.1 When boot does not reach login: the rescue shell

The most important thing to know is that QSOE does not simply hang when it cannot finish starting. If `init` cannot mount the root filesystem — there is no disk, or the disk is empty or unformatted — it does not stall waiting for one. It prints a line saying so and drops you straight into an interactive shell from the boot image:

```
[init] root filesystem unavailable -- entering interactive shell.  
      Use 'shutdown' to power off.
```

This rescue shell is a real `qsh`, so you can look around and work out what happened. What you have is the boot image only: the commands in `/bin` (§5.6) and the `/sys` and `/proc` views all work, but `/usr` is *not* mounted, so the programs that live there — and anything under `/etc` or `/home`, which are symlinks into `/usr` — are not available. That is exactly the situation to diagnose: read `/sys` to confirm what booted, check the log (§8.2), and when you are done, `shutdown`.

Why no login here. `login` and `getty` live on the disk, under `/usr/sbin`, precisely because a system that cannot mount `/usr` has nothing to log in to. The rescue shell is the deliberate, useful fallback rather than a failure.

8.2 Reading the system log

When something does not start as expected, the system log is the first place to look. The logging service starts first in boot and collects events from the kernel and from every server; `sloginfo` (§5.4) prints them:

```
[/ ]# sloginfo
```

Each line carries a timestamp, a severity (from `CRIT` and `ERROR` down through `WARN` and `INFO` to debug levels), the subsystem that reported it, and the message. Scanning for the `ERROR` and `WARN` lines around the point a boot stopped usually shows which driver or server did not come up, and why.

8.3 Common early problems

A few problems account for most of what goes wrong on a first run under QEMU (Table 8.1).

Symptom	Most likely cause and fix
QSOE/N boots, then hangs after starting the NVMe driver; no login.	QEMU older than 11.0.1: the NVMe interrupt never arrives (§2.2). Use QEMU 11.0.1 or newer, or set <code>QEMU=</code> to a current build.
Nothing appears on the console at all.	The firmware or launch is wrong — check that the edk2 firmware is installed and found (§2.2), and prefer <code>run-nvme.sh</code> over a hand-written QEMU line (§2.3).
Boot reaches the <i>rescue shell</i> instead of a login.	The root filesystem did not mount: an empty or missing disk image, or the wrong companion image for the variant (§2.4). Check <code>/sys/cmdline</code> and the log (§8.2).
Login is refused for a user you know.	Check the password (Table 3.1); after a wrong attempt <code>login</code> pauses briefly before re-prompting, which is normal.

Table 8.1: Common early problems and their usual causes.

When in doubt, the two facts that resolve most confusion are the QEMU version (§2.2) and which variant you actually booted (`/sys/osname`).

8.4 Shutting down

To bring the system down, use `shutdown` to power off or `reboot` to restart (§5.5); both require `root`:

```
[/]# shutdown
```

This is the clean way to stop: it asks the platform to power the machine off, and under QEMU that ends the emulator. Shutting down properly rather than just closing the window is the right habit for when the filesystem becomes writable (§6.3) and an orderly stop will matter; on the read-only system of 0.1 nothing is lost either way, but the habit is worth keeping.

Forcing the emulator to quit. If a run is wedged and `shutdown` is not reachable, you can still quit QEMU itself: with the console on `stdio` as `run-nvme.sh` sets it up, press `Ctrl-a` then `x` to terminate the emulator. That is an emulator escape hatch, not a QSOE feature — on real hardware you power-cycle the board.

8.5 Where to go next

That completes the tour: you can obtain and run QSOE, log in, use the shell and the commands, find your way around the filesystem, and bring the system down again. For *why* the system is built the way it is — the two kernels, the single userspace, the message-passing design behind everything in this guide — see the *Design & Architecture* guide (QSOE-DSN). For writing programs and resource servers of your own, see the *Programming Book* (QSOE-PRG) and, for bringing existing software across, the *Application Porting Guide* (QSOE-APG).